PROJECT TITAN

Ready-to-Use Build Prompts

Multi-Agent Autonomous Software Engineering Platform

Specification Series — Draft v1.0

2026-07-04

PROJECT TITAN — Ready-to-Use Build Prompts

These are the copy-paste prompts for actually **building** TITAN with Claude Code, one phase at a time. Each prompt tells the AI exactly which specification volume to implement and what "done" means.

How to use (step by step):

1. Create an empty folder for the project, e.g. `D:\titan`.
2. Copy this `Project-TITAN-Docs` folder inside it, so the specs live at `D:\titan\Project-TITAN-Docs\`.
3. Open Claude Code in `D:\titan`.
4. Paste **Prompt 1**. Let it finish completely. Check the result runs.
5. Only then paste Prompt 2, and so on, in order. Never skip a phase.
6. If a phase fails or looks wrong, don't move on — paste the Fix-It prompt (at the bottom) instead.

Each phase builds on the previous one, matching the dependency order of the specs (foundation → backend → agents → LLM gateway → memory → workflow → dashboard → security → production).

Prompt 1 — Project Scaffold & Foundation

```
You are implementing PROJECT TITAN, a multi-agent autonomous software engineering
platform. The complete specification is in ./Project-TITAN-Docs/ — treat it as the
single source of truth for every decision.

Read these two documents fully before writing any code:
- Project-TITAN-Docs/01-Volume-1-Foundation-and-Product-Requirements.md
- Project-TITAN-Docs/03-Volume-3-Backend-Engineering-and-Infrastructure.md (section on repo structure and Docker
Compose)

Then build PHASE 1 — the project skeleton:
1. Create the monorepo structure exactly as specified (apps/frontend Next.js 15 +
   TypeScript + Tailwind + shadcn/ui; apps/backend Python 3.12 + FastAPI + SQLAlchemy
   + Pydantic v2; packages/shared for shared types; infra/ for Docker).
2. Docker Compose file that starts: PostgreSQL 16, Redis 7, Qdrant, the backend, and
   the frontend — all runnable locally with free tools only.
3. Backend: FastAPI app factory, settings via pydantic-settings from .env, health
   endpoint (/api/v1/health) that reports DB/Redis/Qdrant connectivity, structured
   JSON logging, Alembic set up for migrations.
4. Frontend: Next.js app with dark-mode layout shell, sidebar navigation with the 12
   screens named in Volume 4 (pages can be empty stubs labelled "coming in later
   phase" — that is allowed ONLY for pages, never for backend code).
5. A root README explaining how to run everything (docker compose up).

Definition of done: `docker compose up` starts cleanly; the health endpoint returns
all services healthy; the frontend loads and shows the navigation shell. Write tests
for the health endpoint. No TODOs in backend code.
```

Prompt 2 — Database Schema & Core API

```
Continue PROJECT TITAN. Specs are in ./Project-TITAN-Docs/ (source of truth).

Read fully: Project-TITAN-Docs/03-Volume-3-Backend-Engineering-and-Infrastructure.md

Build PHASE 2 — the data layer and core REST API:
1. Implement the complete PostgreSQL schema from Volume 3 as SQLAlchemy models +
   Alembic migrations: users/roles, projects, milestones, sprints, agents, tasks,
   task_dependencies (with cycle check), runs, artifacts, messages, decisions, and
   the transactional outbox table.
2. Implement the REST API conventions from Volume 3: UUIDv7 ids, cursor pagination,
   RFC 9457 problem+json errors, ETag optimistic concurrency.
3. CRUD endpoints for projects, tasks (with dependency management), agents, and
   messages, matching the endpoint map and example payloads in Volume 3.
4. Authentication: OAuth2/OIDC + JWT as specified, with the RBAC roles and permission
   matrix from Volume 3. Seed a default admin user for local dev.
5. Rate limiting (token bucket) and the caching tiers described in the spec.

Definition of done: alembic upgrade head runs clean on a fresh DB; every endpoint has
integration tests (happy path + auth failure + validation failure); the task
dependency cycle check is unit-tested.
```

Prompt 3 — Agent Framework & Message Protocol

```
Continue PROJECT TITAN. Specs are in ./Project-TITAN-Docs/ (source of truth).

Read fully: Project-TITAN-Docs/02-Volume-2-Multi-Agent-Intelligence.md

Build PHASE 3 — the multi-agent framework (no LLM calls yet; the gateway comes in
Phase 4 — use a mock provider for tests):
1. The declarative AgentDefinition model (YAML + Pydantic) from Volume 2, and load
   the full 16-agent roster (CEO, Product Manager, Project Manager, Orchestrator/
   Planner, System Architect, Backend Engineer, Frontend Engineer, Database Engineer,
   DevOps, QA, Security, Code Reviewer, Critic, Documentation, Research, Memory) with
   their roles, authority levels, and tool allow-lists as data files.
2. The 8-type structured message protocol (TaskAssignment, ProgressReport,
   ReviewRequest, ArchitectureDiscussion, BugReport, DecisionLog, MeetingSummary,
   ConflictResolution) with the common envelope, Pydantic-validated, persisted to
   PostgreSQL via the outbox and delivered over Redis Streams with at-least-once,
   idempotent consumption exactly as Volume 2 specifies.
3. The agent runtime turn model: hydrate context → call provider (mock for now) →
   act (emit messages / update tasks), stateless between turns.
4. The conflict-resolution state machine and escalation ladder from Volume 2.

Definition of done: an integration test drives a scripted two-agent exchange
(TaskAssignment → ProgressReport → ReviewRequest → DecisionLog) end-to-end through
Redis Streams with all messages persisted and replayable; duplicate delivery is
proven idempotent by test.
```

Prompt 4 — LLM Gateway & Tool Layer

```
Continue PROJECT TITAN. Specs are in ./Project-TITAN-Docs/ (source of truth).

Read fully: Project-TITAN-Docs/07-Volume-7-Integrations-and-Tool-Ecosystem.md

Build PHASE 4 — the multi-LLM provider gateway and the agent tool layer:
1. The provider-agnostic LLM gateway from Volume 7: unified chat/completion
   interface; adapters for Anthropic, OpenAI, Google, and Ollama (local); models.yaml
   per-agent-role routing; circuit-breaker fallbacks; token + cost accounting stored
   per run; hard budget caps. Switching providers must be config-only. Include the
   zero-API-key Ollama-only profile so everything runs free and local.
2. The tool abstraction layer: namespaced JSON-Schema tool registry, per-agent
   allow-list enforcement (from Phase 3), risk gates (read/write/admin with human
   approval for admin), and the immutable audit log of every tool call.
3. Core tools for MVP: fs (workspace-scoped), git, terminal (argv-only, sandboxed,
   no shell interpolation), and web research (pluggable, off by default).
4. Wire the Phase-3 agent runtime to the real gateway: one demo flow where the CEO
   agent receives a project brief and produces user stories via a real local model
   through Ollama.

Definition of done: gateway unit tests run against a fake provider; the Ollama
profile is integration-tested (skippable if Ollama absent); every tool call appears
in the audit log with agent id, arguments hash, and outcome; budget cap abort is
tested.
```

Prompt 5 — Memory, Knowledge & RAG

```
Continue PROJECT TITAN. Specs are in ./Project-TITAN-Docs/ (source of truth).

Read fully: Project-TITAN-Docs/05-Volume-5-AI-Brain-Memory-Knowledge-RAG.md

Build PHASE 5 — the AI brain:
1. The memory taxonomy from Volume 5: working memory in Redis; episodic, semantic,
   and procedural long-term memory canonical in PostgreSQL with Qdrant as the derived
   vector index, fed through the outbox + Redis Streams ingestion pipeline.
2. Embedding pipeline: content-type-aware chunking, nomic-embed-text via Ollama
   through the Phase-4 gateway (config-swappable to cloud embedders).
3. Hybrid retrieval: Qdrant dense + PostgreSQL full-text fused with reciprocal rank
   fusion, optional reranker, assembling the typed cited Memory Packet per agent call
   with the context-window budget rules from Volume 5.
4. Knowledge base tables (coding standards, patterns, templates, decision history)
   with proposal/approval writes; knowledge-graph nodes/edges in PostgreSQL.
5. Hook memory into the agent runtime: every agent turn reads a Memory Packet and
   writes episodic records; reflection records persist after task completion.

Definition of done: a retrieval eval test seeds known memories and asserts the right
ones come back top-k; the ingestion pipeline is proven idempotent; an agent turn
integration test shows memory read + write round-trip.
```

Prompt 6 — Autonomous Workflow Engine

```
Continue PROJECT TITAN. Specs are in ./Project-TITAN-Docs/ (source of truth).

Read fully: Project-TITAN-Docs/06-Volume-6-Engineering-Workflow-and-Standards.md

Build PHASE 6 — the pipeline that turns an idea into software:
1. The 12-handoff delivery pipeline from Volume 6 (User → CEO → PM → Project Manager
   → Architect → Engineers → QA → Security → Code Reviewer → DevOps → Documentation →
   Release) as a persisted state machine: every handoff has its trigger, versioned
   Pydantic artifact, acceptance gate, and bounded failure path (3 retries then human
   escalation).
2. The per-component self-improvement loop (Implementation → Review → Critique →
   Refactor → Testing → Security Scan → Performance Analysis → Approval → Merge) with
   the hard 3-cycle cap and convergence rules from Volume 6.
3. CEO clarification policy: max 5 essential questions in one batch; everything else
   becomes logged, user-overridable assumptions.
4. Task orchestration: dependency-driven readiness, race-safe assignment, sprint and
   milestone mechanics, automatic blocker detection.
5. The six auto-generated artifacts (roadmap, kanban data, dependency graph, release
   plan, risk analysis, tech-debt report) as derived views over the event log,
   exposed via API endpoints.

Definition of done: an end-to-end test (mock LLM) runs a tiny project brief through
the entire pipeline to a simulated release, with every handoff, gate decision, and
loop iteration visible in the database; the 3-cycle cap and escalation paths are
tested.
```

Prompt 7 — Dashboard Frontend

```
Continue PROJECT TITAN. Specs are in ./Project-TITAN-Docs/ (source of truth).

Read fully: Project-TITAN-Docs/04-Volume-4-Frontend-and-Dashboard.md

Build PHASE 7 — the full dashboard, replacing the Phase-1 stubs:
1. Implement every screen specified in Volume 4: Organization Overview, Projects
   (list + detail), Kanban board, Task/Dependency Graph view, Agent Status grid, Live
   Agent Conversations, Logs viewer, Architecture Viewer, Deployment Dashboard,
   Monitoring, Memory Explorer, Plugin Manager, Settings — with the layouts, states
   (loading/empty/error), and interactions the spec defines.
2. Realtime: WebSocket sync from the backend event stream so agent activity, task
   moves, and messages appear live.
3. Design system: Tailwind + shadcn/ui tokens from the spec, dark mode default with
   light mode toggle, responsive breakpoints, WCAG 2.1 AA basics, keyboard shortcuts
   and the notification system from Volume 4.
4. State: server components + React Query + WebSocket cache sync as specified.

Definition of done: every screen renders real backend data (seeded demo project);
the live conversation feed updates without refresh during a pipeline run; Playwright
e2e tests cover the critical journeys (create project → watch pipeline → approve
release).
```

Prompt 8 — Security Hardening

```
Continue PROJECT TITAN. Specs are in ./Project-TITAN-Docs/ (source of truth).

Read fully: Project-TITAN-Docs/08-Volume-8-Security.md

Build PHASE 8 — implement the security specification:
1. Work through every MUST requirement (TITAN-SEC-*) in Volume 8 and implement or
   verify each one, keeping a checklist file mapping requirement id → status → where
   implemented.
2. Priorities: agent/code-execution sandboxing (container isolation, resource
   limits, network policy for generated code), prompt-injection defenses at every
   agent input boundary, secret management (secrets never in prompts, logs, or LLM
   context — enforced by test), input validation everywhere, immutable audit logging,
   rate limiting and abuse prevention.
3. Add the OWASP Top 10 + OWASP LLM Top 10 mapping from the spec as automated checks
   where possible (dependency scanning, SAST) wired into the test suite.
4. Anything that cannot be fully implemented locally must fail closed and be logged
   in the checklist as a documented gap — never silently skipped.

Definition of done: the checklist file shows every MUST either implemented-with-test
or documented-gap-with-reason; security tests pass; a test proves a seeded secret
never appears in any prompt, log line, or LLM request payload.
```

Prompt 9 — CI/CD, Observability & Production Packaging

```
Continue PROJECT TITAN. Specs are in ./Project-TITAN-Docs/ (source of truth).

Read fully: Project-TITAN-Docs/09-Volume-9-Production-and-Operations.md

Build PHASE 9 — production readiness:
1. GitHub Actions pipeline from Volume 9: lint → typecheck → tests → build → scan →
   publish images (digest-pinned) — with the deploy stages defined even if cloud
   deploy stays unconfigured locally.
2. Observability: Prometheus metrics including the agent-level metrics (tokens, USD
   cost, task throughput, queue age), the 5 Grafana dashboards provisioned as code,
   Loki structured logs, OpenTelemetry tracing propagated across Redis Streams —
   all added to Docker Compose so it runs locally.
3. Kubernetes manifests from Volume 9 (deployments, services, ingress, autoscaling)
   in infra/k8s, validated with kubeconform or equivalent — deployable later, correct
   now.
4. Backup/restore scripts for PostgreSQL (WAL/PITR approach from the spec) with a
   restore-verification script.
5. The production readiness checklist from Volume 9 as a living document with each
   item checked or marked with its gap.

Definition of done: CI workflow passes locally via act or is fully lint-validated;
docker compose up includes the monitoring stack and dashboards show live agent
metrics during a demo pipeline run; backup + restore round-trip is tested.
```

Prompt 10 — Evolution Features (Optional / Later)

```
Continue PROJECT TITAN. Specs are in ./Project-TITAN-Docs/ (source of truth).

Read fully: Project-TITAN-Docs/10-Volume-10-Future-and-Autonomous-Evolution.md

Build PHASE 10 — the self-improvement layer (only after Phases 1–9 are stable):
1. Per-task agent scorecards (the 8-dimension SQL schema from Volume 10) rolling up
   into the per-agent KPIs defined in Volume 2.
2. Post-task reflection and post-project retrospective records persisted to memory
   (Phase 5) automatically at pipeline completion.
3. The failure taxonomy and root-cause records, feeding Evolution Proposals: every
   proposed prompt/routing/policy change is expressed as a Git PR under /evolution/**
   and requires human approval — never auto-applied.
4. TITAN-Bench: the offline benchmark suite from Volume 10 that scores agent
   performance so prompt/routing changes can be A/B evaluated before approval.
5. The multi-model debate protocol for high-stakes decisions, config-gated off by
   default.

Definition of done: after a demo project completes, scorecards and a retrospective
exist in the database; a sample evolution proposal PR is generated end-to-end and
correctly blocks without human approval; TITAN-Bench runs against the mock provider.
```

Fix-It Prompt (use whenever a phase went wrong)

```
The last phase of PROJECT TITAN has problems. Before writing any new code:
1. Re-read the specification volume for this phase in ./Project-TITAN-Docs/ and list
   every requirement it defines for what we just built.
2. Audit the current code against that list: for each requirement mark OK, BROKEN, or
   MISSING, with file references.
3. Show me the audit table first. Then fix everything BROKEN or MISSING, in
   dependency order, running the tests after each fix.
4. Never delete working functionality to make a test pass; fix the root cause.
Here is what I observed going wrong: <describe the problem here>
```

Part of the PROJECT TITAN specification set — see `00-README-Master-Overview.md` for the volume index.